

Base de conocimiento para software robusto

Resumen, evaluación y prompt para proyectos nuevos · 2026-10-07 · D:\BaseConocimiento

1. Qué es y cómo está organizada

Una carpeta única que Claude lee al empezar un proyecto, para no repetir errores ya cometidos. Va de lo general a lo concreto:

Carpeta	Contenido
01-fundamentos	34 lecciones (error → regla → cómo verificarlo) y guía de 7 fases con criterio de terminado
02-arquitectura	Clean/Hexagonal, SOLID, backend por tecnología, API y seguridad (OWASP)
03-base-de-datos	Elección de motor, transacciones/ACID, ABM robusto, esquema, migraciones y respaldos
04-plantillas	CLAUDE.md, prompt inicial, ADR, documento de cambio
05-referencia	Código real verificado del proyecto San Andrés (799 tests) y su auditoría
06-pruebas	Norma de testing/TDD y contraste con el proyecto
07-grafos	Mapas de relaciones generados con graphify

2. Opinión

- **Lo mejor:** está basada en errores reales, no en teoría; cada regla tiene un test que la verifica; hay código ejecutable de referencia; el orden de lectura es claro; y el README admite honestamente lo que NO está probado.
- **Diseño correcto:** el CLAUDE.md global es corto y apunta a la base; las reglas del proyecto mandan sobre las genéricas; hay un ciclo de mejora (cada corrección manual se vuelve lección).
- **Riesgo principal:** sesgo. Todo sale de un proyecto (Next.js + Prisma + SQLite). Lo demás es investigación sin validar en la práctica.
- **Detalles a corregir:** la guía salta del paso 3 al 5 en "Al cerrar cada bloque"; el README dice "15 commits" pero hay 17 documentos de auditoría; los grafos no incluyen los documentos nuevos.

3. ¿Falta profundizar? Sí, en estas áreas (por prioridad)

- **Despliegue y operación:** Docker, CI/CD real (el CI aún no corrió en GitHub), variables de entorno, logs, monitoreo, respaldos restaurados de verdad.
- **Un segundo stack probado:** PostgreSQL con concurrencia real (FOR UPDATE, SKIP LOCKED, versión de fila, Outbox, idempotencia) y un backend en C# o Python.
- **Frontend y accesibilidad:** estados de carga/error, formularios, e2e por clics en todas las pantallas, rendimiento en móvil.
- **Seguridad aplicada:** pruebas de penetración básicas, dependencias vulnerables, manejo de archivos subidos, secretos.
- **Cumplimiento por país:** facturación electrónica y protección de datos (depende del cliente).
- **Evolución de datos en producción:** migraciones sin parar el servicio, volumen grande, índices y planes de consulta.

4. Laboratorios realizados (2026-10-07)

Cuatro proyectos pequeños en 08-laboratorios/, con 58 tests propios en verde. Cada uno dejó lecciones (L-060 a L-072):

- **Lab 1 · PostgreSQL 18 real (12 tests):** cobros simultáneos, sin sobreventa, lost update (error y arreglo), versión de fila, idempotencia, Outbox, SKIP LOCKED, write skew con SERIALIZABLE, deadlocks.
- **Lab 2 · C# / .NET 8 (13 tests):** capas Dominio, Aplicación, Infraestructura y API contra PostgreSQL real. Un test descubrió que un ejemplo numérico del prompt estaba mal (\$3.376,35 es lo correcto).
- **Lab 3 · Respaldo y pipeline (5 tests):** respaldo lógico con manifiesto y SHA-256, respaldo físico en frío, restauración tras un desastre simulado y pipeline local en verde. **GitHub Actions y Docker están escritos pero NO ejecutados** (sin remoto ni Docker).
- **Lab 4 · Frontend y seguridad (28 tests):** e2e en escritorio y móvil, axe, teclado, XSS, CSRF, cabeceras, límites y 429. Encontró 4 errores reales de la propia app.
- **Siguen sin probarse:** Python, Java y Go; pg_dump y recuperación a un punto en el tiempo; pruebas de penetración reales; despliegue en la nube.

Además se documentaron los plugins instalados y cuándo usarlos en 01-fundamentos/HERRAMIENTAS-Y-PLUGINS.md.

5. Prompt para próximos proyectos

Copiar, completar y pegar. Lo que quede vacío, Claude lo pregunta (máx. 4 preguntas, una sola tanda). Antes: copiar CLAUDE.md.plantilla como CLAUDE.md y 01-TESTING-guia-tdd.md como TESTING.md.

Quiero construir [NOMBRE]: [que hace y para quien, en una frase].

USUARIOS Y ROLES

- [rol]: puede [...]

ALCANCE DE LA V1

Si: [funciones] No (por ahora): [funciones]

REGLAS DE NEGOCIO (con ejemplos numericos)

- [Ej.: 3 x \$1.250,50 con 10 % de descuento = \$3.376,35]
- [Ej.: el stock se descuenta al cobrar y se devuelve al anular]

DATOS

- Sensibles: [dinero / stock / personales]
- Datos a migrar: [si/no, formato] Volumen a 1 y 3 años: [...]

DONDE CORRE

- [escritorio / LAN / nube], [N] usuarios simultaneos, [con/sin internet]
- Base de datos y stack: [a tu criterio con justificacion / fijo: ...]

EQUIPO

- Quien hace el frontend y que carpetas toca cada uno.

COMO TRABAJAR

- Lee primero D:\BaseConocimiento\README.md, LECCIONES-APRENDIDAS.md y GUIA-APP-ROBUSTA.md.
- Sigue las fases 0 a 6; no pases de fase sin cumplir su criterio.
- Elige la base de datos con 03-base-de-datos/01 y registrala en un ADR.
- Por funcionalidad: contrato de API -> tests -> implementacion -> pantalla -> documento corto.
- Transacciones atomicas; ABM completo (paginado, version de fila, baja logica o anulacion).
- Cierra cada bloque con tests, build, linter y tipos verdes, mostrando la salida.
- Al final: lista de pendientes y de lo que no pudiste verificar; agrega a la base las lecciones nuevas.

Variantes cortas: *Funcionalidad nueva:* "Agrega [X]. Lee CLAUDE.md, las lecciones relevantes y el código a tocar; resume el diseño en 5 líneas y espera mi OK si hay alternativas." · *Auditoría:* "Audita el backend con 03-base-de-datos/02 y 02-arquitectura/03; hallazgos por severidad, un commit por grupo, test REGRESIÓN por arreglo."

Claves de un buen prompt: reglas con números, alcance cerrado, restricciones reales, criterio de terminado verificable, y quién toca qué.